Practically Groovy: Mark it up with Groovy Builders

Side-step the details of markup languages and focus on your application content instead

Skill Level: Intermediate

Andrew Glover (andrew@thirstyhead.com)
Co-Founder
ThirstyHead.com

Scott Davis (scott@thirstyhead.com)
Founder
ThirstyHead.com

12 Apr 2005

Groovy Builders let you mimic markup languages like XML, HTML, Ant tasks, and even GUIs with frameworks like Swing.They're especially useful for rapid prototyping and, as *Practically Groovy* columnist Andrew Glover shows you this month, they're a handy alternative to data binding frameworks when you need consumable markup in a snap!

A few months ago when I first wrote about Ant scripting with Groovy, I mentioned the notion of *Builders* in Groovy. In that article I showed you how easy it was to build expressive Ant build files with a Groovy class called `AntBuilder`. In this article, I'll dig deeper into the world of Groovy Builders to show you what else you can do with these powerful classes.

Building with Builders

Groovy's Builders let you effortlessly mimic markup languages like XML, HTML, Ant tasks, and even graphical user interfaces with frameworks like Swing. With a builder, you can quickly create a sophisticated markup like XML without having to deal with XML itself.

The Builder paradigm turns out to be quite simple. Methods attached to an instance of a Builder represent an element of that markup (such as a <body> tag in HTML). Objects created inside of closures attached to methods represent child nodes (for instance, a <p> tag inside of a wrapping <body> tag).

So that you can see this in action, I'll create a simple Builder to programmatically represent an XML document that has the structure shown in Listing 1.

About this series

The key to incorporating any tool into your development practice is knowing when to use it and when to leave it in the box. Scripting languages can be an extremely powerful addition to your toolkit, but only when applied properly to appropriate scenarios. To that end, *Practically Groovy* is a series of articles dedicated to exploring the practical uses of Groovy, and teaching you when and how to apply them successfully.

Listing 1. A simple XML structure

```
<person>
  <name first="Megan" last="Smith">
    <age>32</age>
    <gender>female</gender>
  </name>
  <friends>
    <friend>Julie</friend>
    <friend>Joe</friend>
    <friend>Hannah</friend>
  </friends>
</person>
```

Representing this structure is quite simple. I first attach a `person` method to a Builder instance, which now represents the root node, <person>, of the XML. To create child nodes, I create a closure and declare a new object dubbed `name`, which takes parameters in the form of a `map`. Those parameters, incidentally, form the basis of attributes attached to an element.

Next, within the `name` object, I attach a closure with two additional objects, `age` and `gender`, which correspond to similar child elements of <name>. Are you getting the hang of this yet? It's pretty easy.

The <friends> element is a sibling of <person>, so I jump out of the closure, declare a `friends` object and, of course, attach a closure that contains a collection of `friend` elements, as shown in Listing 2.

Listing 2. Builders are so simple

```
class XMLBuilder{
  static void main(String[] args) {
```

```
    def writer = new StringWriter()
    def builder = new groovy.xml.MarkupBuilder(writer)
    def friendnames = [ "Julie", "Joey", "Hannah"]

    builder.person() {
       name(first:"Megan", last:"Smith") {
         age("32")
         gender("female")
       }
       friends() {
         for (e in friendnames) { friend(e) }
       }
    }
    println writer.toString()
  }
}
```

As you can see, the Groovy representation is quite elegant and easily maps to the corresponding markup representation. Underneath, Groovy is obviously handling the tedious markup elements (like < and >), allowing me to focus more on the content and not so much on the details of the structure.

Show me the HTML

Builders can also facilitate building HTML, which can come in handy when developing Groovlets. As a brutally simple case, let's imagine I wanted to create a simple HTML page like the one found in Listing 3:

Listing 3. HTML 101

```
<html>
 <head>
  <title>Groov'n with Builders</title>
 </head>
 <body>
  <p>Welcome to Builders 101. As you can see this Groovlet is fairly simple.</p>
 </body>
</html>
```

I could easily code it in Groovy, as shown in Listing 4:

Listing 4. HTML in Groovy 101

```
class HTMLBuilderExample{
  static void main(String[] args) {
    def writer = new StringWriter()
    def builder = new groovy.xml.MarkupBuilder(writer)
    builder.html(){
      head(){
        title("Groov'n with Builders"){}
      }
      body(){
        p("""Welcome to Builders 101. As you can see
             this Groovlet is fairly simple.""")
```

```
      }
    }
    println writer.toString()
  }
}
```

For a little more fun, let's see how easy it is to build a full-fledged GUI with Builders. I mentioned earlier that Groovy's `SwingBuilder` makes it possible to construct GUIs in an extremely simple manner. You can see `SwingBuilder` at work in Listing 5.

Listing 5. GUI Builders in Groovy are GROOVY

```
import java.awt.FlowLayout
import javax.swing.*
import groovy.swing.SwingBuilder

class SwingExample{
  static void main(String[] args) {
    def swinger = new SwingBuilder()
    def langs = ["Groovy", "Ruby", "Python", "Pnuts"]

    def gui = swinger.frame(title:'Swinging with Groovy!', size:[290,100]) {
      panel(layout:new FlowLayout()) {
        panel(layout:new FlowLayout()) {
          for (lang in langs) {
            checkBox(text:lang)
          }
        }
        button(text:'Groovy Button', actionPerformed:{
          swinger.optionPane(message:'Indubitably Groovy!').createDialog(null, 'Zen
Message').show()
        })
        button(text:'Groovy Quit', actionPerformed:{ System.exit(0)})
      }
    }
    gui.show()
  }
}
```

And the results are shown in Figure 1. Not bad, eh?

Figure 1. The Zen of GUI programming in Groovy

It's easy to imagine what a powerful tool something like `SwingBuilder` could be for prototyping, isn't it?

Show me something real

So those examples were trivial, but fun. I hope I've made it clear that Groovy's Builders enable you to avoid the underlying markup of a particular language, like XML. Obviously, avoiding XML or HTML sometimes is preferable, and markup facilitators are definitely not new to the Java™ platform; for example, one of my favorite XML facilitating frameworks is JiBX.

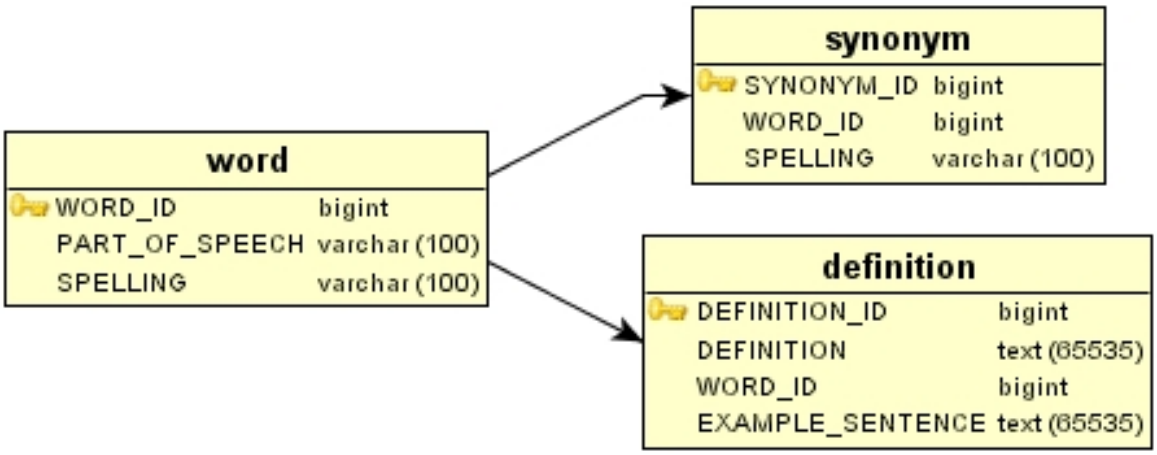
With JiBX, you can easily map an XML structure to an object model or *vice versa*. Binding is a powerful paradigm and there are a plethora of similar tools that do it -- such as JAXB, Castor, and Zeus.

The only thing about binding frameworks is that, well, they *can be* a bit time consuming. Fortunately, you can use Groovy's Builders for a *simpler* solution that's just as effective in some cases.

Pseudo-binding with Builders

Imagine for a moment that you have a simple database representing a dictionary of English words. You have a table for `words`, another for their `definitions`, and lastly a table for `synonyms`. Figure 2 is a simple representation of the database.

Figure 2. A dictionary database



As you can see, the database is quite straightforward: `words` have a one-to-many relationship with `definitions` and `synonyms`.

The dictionary database has a consumer who is looking to ingest an XML structure representing the key aspects of the database's contents. The XML structure being sought is shown in Listing 6.

Listing 6. Ingestible dictionary XML

```
<words>
  <word spelling="glib" partofspeech="adjective">
    <defintions>
      <defintion>Performed with a natural, offhand ease.</defintion>
      <defintion>Marked by ease and fluency of speech or writing that often suggests
          or stems from insincerity, superficiality, or deceitfulness</defintion>
    </defintions>
    <synonyms>
      <synonym spelling="artful"/>
      <synonym spelling="urbane"/>
    </synonyms>
  </word>
</words>
```

If you chose to address this problem using a binding framework like JiBX, you would most likely have to create some intermediary object model to get from your relational model to your final destination of an XML model. You would then have to read the contents of the database into your object model and then request the underlying framework to marshall its internal structure to an XML format.

Implicit in this process is the step also taken to map (using the desired framework's procedures) the object structure to an XML format. Some frameworks, like JAXB, actually generate Java objects for you from the XML and other frameworks, like JiBX, allow you to custom-map your own Java objects to an XML format. Either way though, it can be a lot of work.

And it's a noble effort. I'm not advocating the avoidance of binding frameworks. There, I've said it: You are forewarned. What I plan on showing you is a quicker way to generate that XML.

Consumable XML is a snap

Using Groovy's `MarkupBuilder` combined with your new favorite database access framework, GroovySql, you can easily generate the consumable XML. All you need to do is figure out the required queries and map the results to a Builder instance -- and in a snap you've got your XML document representing the contents of your dictionary database.

Let's walk through this step by step. First, you create an instance of a Builder, which in this case is a `MarkupBuilder` because you intend to generate XML. Your outermost XML element (that is, the root) is `words` so you create a `words` method. In the closure you call your first query and map the results of that query in an iterator to your child node, which is `word`.

Next, you create two child nodes of `word` via two new queries. You create a `definitions` object and map it inside an iterator, then do something similar for `synonyms`.

Listing 7. Putting it all together with Builders

```
class WordsDbReader{
  static void main(String[] args) {
    def sql = groovy.sql.Sql.newInstance("jdbc:mysql://localhost:3306/words", "words",
            "words", "com.mysql.jdbc.Driver")
    def writer = new StringWriter()
    def builder = new groovy.xml.MarkupBuilder(writer)
    builder.words() {
      sql.eachRow("select word_id, spelling, part_of_speech from word"){ row ->
        builder.word(spelling:row.spelling, partofspeech:row.part_of_speech){
          builder.definitions(){
            sql.eachRow("select definition from definition where word_id =
                        ${row.word_id}"){ defrow ->
              builder.definition(defrow.definition)
            }
          }
          builder.synonyms(){
            sql.eachRow("select spelling from synonym where word_id =
                        ${row.word_id}"){ synrow ->
              builder.synonym(synrow.spelling)
            }
          }
        }
      }
    }
    new File("dbouuput.xml").append(writer.toString())
  }
}
```

Conclusion

The binding solution I've shown you here seems almost sinfully easy, especially from a Java-purist point of view. While the solution is no *better* than what you could accomplish with a binding framework like JABX or JiBX, it sure does come together faster -- and I'd argue somewhat easier. Could I have done something similar in *plain Jane* Java code? Sure, but I bet I would have had to deal with the XML at some point along the way.

The speed and simplicity with which you can develop with Groovy's Builders will be beneficial in situations that call for markup; for example, as you saw in the second example, I was able to whip up some XML representing a database in snap. Builders are also a fabulous option when it comes to prototyping, or for when minimal development time and effort are required to produce a working solution.

What do I have for you next month in *Practically Groovy*? Why, using Groovy inside of the Java language, of course!

Downloads

Description	Name	Size	Download method
Sample code	j-pg04125.zip	2KB	HTTP

Information about download methods

About the authors

Andrew Glover
Andrew Glover is a developer, author, speaker, and entrepreneur. He is the founder of the easyb Behavior-Driven Development (BDD) framework and is the co-author of three books: Continuous Integration, Groovy in Action, and Java Testing Patterns. He teaches a wide variety of Groovy-, Grails-, and testing-related classes at ThirstyHead.com. You can keep up with Andy at thediscoblog.com where he routinely blogs about software development.

Scott Davis
Scott Davis is an internationally recognized author, speaker, and software developer. He is the founder of ThirstyHead.com, a Groovy and Grails training company. His books include Groovy Recipes: Greasing the Wheels of Java, GIS for Web Developers: Adding Where to Your Application, The Google Maps API, and JBoss At Work. He writes two ongoing article series for IBM developerWorks: Mastering Grails and Practically Groovy.